10 — X (Twitter) Official Alert Source via twitterapi.io (2026-10-14)

Motivation. FYP2 priority #3 (real-time official alerts) and thesis limitation §5.4 (closed social platforms). Sarawak Energy posts disruption notices on X (`@1SarawakEnergy`); Sarawak Water regional accounts (North/South) are **Facebook-only** and remain covered by the telegram + news legs — recorded as a continuing limitation.

Option comparison (checked 2026-10-14)

Option	Est. cost for this use case	Verdict
Official X API (pay-per-use since 2026-02; legacy tiers closed)	~$100/mo at 30-min polling ($0.005/post read)	Rejected: 20x overkill for one quiet handle
Apify + Xquik actor ($0.00015/result)	~$20–35/mo (needs Apify Starter plan)	Rejected: same backend class, extra subscription
twitterapi.io pay-per-use	**~$3–5/mo** (1 handle, 10-min poll, ~$0.00015/tweet, $0.00015/request min)	**Chosen**
Scraping (nitter / twscrape)	free	Rejected 2026-10-13: nitter dead, twscrape unsuitable (doc 04 §2.1)

Chosen trade-off: twitterapi.io is a closed third-party backend (client SDKs are MIT, the scraper is not) — a vendor-lock-in risk of the same class as the lapsed rss.app feed (doc 09). Mitigation: the fetcher is isolated in `utility_alert_reader.py`, spends are hard-capped per poll, and empty key/handles make the source no-op so the poll cycle never depends on it.

Implementation

- `src/core/config.py` — `TWITTERAPI_IO_API_KEY` (secret: `.env` local, Fly secret in prod), `UTILITY_ALERT_X_HANDLES` (default `1SarawakEnergy` — note spelling), `UTILITY_ALERT_X_QUERIES` (comma-separated search phrases, EN+MS)
- `src/scrapers/utility_alert_reader.py` — `fetch_x_alerts()`: searches `GET https://api.twitterapi.io/twitter/tweet/advanced_search` per query (6 s spacing: free tier = 1 req / 5 s, one 429 retry), keeps only tweets whose `author.userName` is a configured handle, max 10/handle/poll, returns `RawAlert(source="x")`; any failure → `[]` (rule #8)
- `src/core/utility_alert_service.py` — `"x"` entry in the poll `fetchers` dict; classifier, `(source, source_ref)` dedup, and delivery staging reused unchanged

Why search instead of the timeline endpoint (verified 2026-10-14): `@1SarawakEnergy`'s profile is login-walled on X, so twitterapi.io's user endpoints (`/twitter/user/info`, `/twitter/user/last-tweets`) return `user not found`, and `from:<handle>` search returns empty — but the search index carries the tweets (`author.userName: "1SarawakEnergy"`). The fetcher therefore searches disruption phrases (which also pre-filters non-alerts and saves per-tweet credits) and filters by author client-side. Trade-off recorded: tweets that match none of the search phrases are invisible to this leg (mitigation: telegram + news legs still catch them).

Cost bound at 1 handle × 10-min poll × 4 queries × max 10 tweets: ≤ ~17,000 requests + ≤ ~43,000 tweet reads/month → under ~$8/month worst case; realistic (sparse official account) ~$3–5/month. Dashboard bonus credits (~100K ≈ 6,700 tweets) cover development and the first weeks.

Test evidence

- 2026-10-14: `python -m compileall src` clean. Live smoke test: 10 real disruption tweets fetched from `@1SarawakEnergy` (e.g. "SRI AMAN: Faulty aeriel cable cause of supply interruption…"), timestamps parsed, rule classifier labelled them `power/restored` / `power/unscheduled` with locations (sri aman, kuching, betong). Credit spend for the whole test: well under 1,000 credits.
- 2026-10-14 production: poll log shows `'x': 10` — 10 alerts stored, only the ≤36 h-old tweet is delivery-eligible (past-window guard works). Note: `TWITTERAPI_IO_API_KEY` must be a **Fly secret**; a leading-whitespace line in `.env` silently produced an empty secret on the first attempt (extract/verify length before `secrets set`).

Reproduce

```
# .env
TWITTERAPI_IO_API_KEY=<key from dashboard>
python -c "from src.scrapers.utility_alert_reader import fetch_x_alerts;
print(len(fetch_x_alerts(['1SarawakEnergy'])), 'tweets fetched')"
```

Production: `flyctl secrets set TWITTERAPI_IO_API_KEY=<key> -a apanewsharitok`, then confirm the `[utility-alert] poll` log shows `'x': N` in `sources`.